

APIs restful

M8: IMPLEMENTACIÓN DE API BACKEND NODE EXPRESS

|**AE1:** CARACTERIZAR LOS ELEMENTOS FUNDAMENTALES DE UNA ARQUITECTURA REST DISTINGUIENDO BUENAS PRÁCTICAS PARA EL DISEÑO DE UNA API REST PARA LA INTEROPERACIÓN DE SISTEMAS.

Introducción

En el desarrollo moderno de software, las APIs RESTful se han convertido en una piedra angular para la comunicación entre aplicaciones. Una API (Interfaz de Programación de Aplicaciones) permite que distintos sistemas se comuniquen entre sí, compartan datos y funcionalidades de forma estructurada. REST (Representational State Transfer) es un estilo de arquitectura que promueve un conjunto de principios para construir servicios web simples, escalables y mantenibles.

Este manual introduce los fundamentos de las APIs RESTful, sus características principales, buenas prácticas de diseño, y ejemplos concretos en JavaScript utilizando Node.js y Express. A través de este recorrido, desarrollarás una comprensión sólida de cómo estructurar y consumir APIs de manera profesional.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender qué es una API RESTful y para qué se utiliza.
- Identificar las características que definen una arquitectura REST.
- Aplicar buenas prácticas en el diseño de rutas, nombres de recursos, y uso de verbos HTTP.
- Utilizar códigos de estado HTTP apropiadamente.
- Implementar endpoints RESTful en Node.js con el framework Express.

Tema 1: ¿Qué es una API RESTful?

¿Qué es una API?

Una API (Application Programming Interface) es un conjunto de reglas que permite que una aplicación se comuniquen con otra. Por ejemplo, cuando una app del clima consulta información de una base de datos meteorológica, lo hace a través de una API.

¿Qué significa REST?

REST (Transferencia de Estado Representacional) es un estilo arquitectónico para diseñar servicios web. Fue definido por Roy Fielding en el año 2000. Una API RESTful es una API que cumple con los principios de REST:

- Utiliza el protocolo HTTP.
- Opera sobre recursos identificables mediante URLs.
- Utiliza verbos HTTP para definir las acciones: GET, POST, PUT, DELETE, PATCH.
- Es stateless: cada petición contiene toda la información necesaria.

REST se basa en una serie de principios que permiten que los sistemas se comuniquen entre sí de forma eficiente y escalable, utilizando el protocolo HTTP.

¿Para qué sirve una API RESTful?

Una API RESTful sirve para construir sistemas que pueden comunicarse entre sí de forma estandarizada. Es ideal para separar el backend del frontend y permite que múltiples clientes (como aplicaciones web, móviles o de escritorio) consuman los mismos datos.

Sirve para construir sistemas que se puedan comunicar entre sí de manera estandarizada. Por ejemplo, un frontend en React puede hacer peticiones a una API RESTful construida en Node.js para mostrar usuarios, productos o publicaciones.

Ejemplo de uso típico:

Una app móvil de compras se comunica con un servidor usando una API RESTful para:

- Obtener productos (GET /productos)
- Realizar una compra (POST /ordenes)
- Consultar el estado de una orden (GET /ordenes/123)

Ventajas de REST

- Simple de entender y usar.
- Compatible con cualquier cliente HTTP (navegador, app móvil, etc.).
- Escalable y flexible.

Desventajas

- Puede volverse complejo al manejar relaciones de recursos o seguridad avanzada.
- No está pensado para conexiones en tiempo real.

Características principales de REST

Una arquitectura RESTful debe cumplir con una serie de **principios o restricciones** que la hacen eficiente, escalable y sencilla de mantener:

1. Interfaz uniforme

Todos los recursos de la API deben ser accedidos de forma coherente. Esto implica:

- Uso estandarizado de URLs para identificar recursos:
GET /usuarios/5 debe devolver siempre el mismo tipo de respuesta.
- Las respuestas deben ser **autodescriptivas**, es decir, explicar por sí solas qué contienen.

2. Peticiones sin estado (stateless)

Cada solicitud HTTP enviada al servidor debe contener **toda la información necesaria** para procesarla. El servidor **no almacena información** de sesiones entre peticiones.

Esto facilita la escalabilidad, ya que cualquier servidor puede manejar cualquier solicitud.

3. Cacheable

Las respuestas deben indicar si pueden ser almacenadas en caché. Esto mejora el rendimiento y reduce la carga del servidor.

Ejemplo:

El servidor puede responder con un encabezado `Cache-Control: max-age=3600` indicando que el recurso puede guardarse por una hora.

4. Separación cliente-servidor

La API RESTful permite que el cliente (como una app web o móvil) esté completamente separado del servidor. Esto favorece el desarrollo paralelo, la reutilización del código y el mantenimiento.

5. Sistema de capas

La arquitectura REST puede estar compuesta por **múltiples capas** (como proxies, balanceadores, capas de autenticación), sin que el cliente lo perciba. Esto mejora la seguridad y la escalabilidad.

Ventajas y desventajas de REST

● Ventajas

1. Simplicidad

- Usa el protocolo HTTP, ampliamente conocido y soportado.
- No requiere bibliotecas ni herramientas especiales para consumirla.

2. Escalabilidad

- Al ser stateless, es fácil distribuir la carga entre múltiples servidores.

3. Independencia entre cliente y servidor

- Permite desarrollar el frontend y el backend por separado.
- Un mismo backend puede ser consumido por diferentes clientes (web, mobile, etc.).

4. Compatibilidad con múltiples formatos

- Aunque el más común es JSON, también puede usar XML, texto plano, etc.

5. Alta adopción

- REST es un estándar de facto en la industria, por lo que hay muchos recursos, herramientas y buenas prácticas disponibles.

Desventajas

1. Manejo complejo de relaciones

- REST no ofrece una forma nativa de manejar relaciones complejas entre recursos. En esos casos, puede ser menos eficiente que otras alternativas como GraphQL.

2. No es ideal para tiempo real

- No está diseñado para conexiones persistentes como WebSockets. Para apps de mensajería o juegos en línea, hay opciones más adecuadas.

3. Demasiadas llamadas al servidor

- Para obtener datos relacionados (por ejemplo, un usuario con sus publicaciones y comentarios), puede requerir múltiples solicitudes.

Reglas de una arquitectura REST

Para que una API sea considerada **RESTful**, debe seguir una serie de reglas o restricciones arquitectónicas. Estas reglas aseguran coherencia, escalabilidad y una experiencia predecible para los desarrolladores que consumen la API.

Interfaz uniforme con mensajes descriptivos

- Las URLs deben identificar claramente los recursos.
- Los mensajes (especialmente las respuestas) deben ser **autodescriptivos**.
- Se espera un comportamiento predecible en cada endpoint.

Ejemplo:

```
GET /usuarios/10
```

Debe devolver un usuario con ID 10 en formato JSON, incluyendo detalles útiles como enlaces relacionados (si se usa HATEOAS).

2. Peticiones sin estado (stateless)

- Cada solicitud debe incluir toda la información necesaria para procesarla.
- El servidor **no mantiene contexto** entre peticiones.

Ejemplo:

Cada vez que se hace `GET /perfil`, el cliente debe enviar su token de autenticación, ya que el servidor no recuerda quién es el cliente.

3. Cacheable

- Las respuestas deben indicar si pueden ser cacheadas.
- Se mejora la eficiencia reduciendo llamadas repetidas al servidor.

Ejemplo:

```
Cache-Control: max-age=3600
```

Esto indica que el cliente puede guardar la respuesta por una hora.

4. Separación cliente-servidor

- El cliente y el servidor se desarrollan de forma independiente.
- El cliente no necesita saber cómo están implementados los datos en el servidor.

Esto permite cambiar el frontend sin afectar la lógica del backend, y viceversa.

5. Sistema de capas

- Una API REST puede tener múltiples capas intermedias (autenticación, logging, validación) sin que el cliente lo perciba.
- Cada capa cumple un rol y puede evolucionar o escalar de forma independiente.

Buenas prácticas en la creación de una API REST

Aplicar buenas prácticas al diseñar una API RESTful es clave para que sea **entendible, predecible, reutilizable y mantenible**. A continuación, te muestro las más importantes:

1. Versionamiento de la API

Permite hacer cambios sin romper el funcionamiento para quienes ya la usan.

Recomendado: versionar en la URL.

Ejemplo:

```
GET /api/v1/usuarios
GET /api/v2/usuarios
```

2. Uso correcto de los verbos HTTP

Cada verbo tiene una intención clara:

Verbo	Acción
GET	Leer recursos
POST	Crear nuevos recursos
PUT	Reemplazar un recurso existente
PATCH	Actualizar parcialmente un recurso
DELETE	Eliminar un recurso

Ejemplo:

```
GET /productos → lista de productos
POST /productos → crear nuevo producto
GET /productos/3 → producto con ID 3
PUT /productos/3 → actualizar todo el producto
DELETE /productos/3 → eliminar producto
```

```
const express = require('express');
```

```

const app = express();

app.use(express.json());

let usuarios = [{ id: 1, nombre: 'Ana' }];

// GET: obtener todos los usuarios
app.get('/usuarios', (req, res) => {
  res.json(usuarios);
});

// POST: crear un nuevo usuario
app.post('/usuarios', (req, res) => {
  const nuevo = req.body;
  usuarios.push(nuevo);
  res.status(201).json(nuevo);
});

// PUT: actualizar un usuario completo
app.put('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const actualizado = req.body;
  usuarios = usuarios.map(u => u.id === id ? actualizado : u);
  res.json(actualizado);
});

// DELETE: eliminar un usuario
app.delete('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  usuarios = usuarios.filter(u => u.id !== id);
  res.status(204).send();
});

app.listen(3000, () => console.log('Servidor corriendo en puerto 3000'));

```

3. Nombres de recursos claros y en plural

Usar sustantivos (no verbos) y en plural para referirse a colecciones de recursos.

✓ Correcto:

```
GET /Clientes
```

✗ Incorrecto:

```
GET /obtenerClientes
```

4. Códigos de estado HTTP apropiados

Informan al cliente sobre el resultado de su solicitud:

Código	Significado
200	OK (éxito)
201	Created (creación exitosa)
400	Bad Request (petición inválida)
401	Unauthorized (sin autorización)
404	Not Found (no encontrado)
500	Internal Server Error

5. Formato de salida consistente

El formato más usado es **JSON**, porque es ligero, legible y compatible con la mayoría de lenguajes.

Ejemplo:

```
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@example.com"
}
```

```
app.get('/usuario/1', (req, res) => {
  res.status(200).json({
    id: 1,
    nombre: 'Ana',
    email: 'ana@example.com'
  });
});
```

Búsqueda y filtrado en APIs RESTful

Una API REST bien diseñada permite al cliente **buscar, filtrar, ordenar y paginar** resultados usando parámetros de consulta (query parameters). Esto brinda flexibilidad y control sin necesidad de múltiples endpoints.

Filtrado

Permite restringir los resultados en función de ciertos criterios.

Ejemplo:

Obtener todos los productos de la categoría “libros”:

```
GET /productos?categoria=libros
```

2. Búsqueda por texto

Buscar por una coincidencia parcial o exacta, útil para buscadores.

Ejemplo:

Buscar productos cuyo nombre contenga “node”:

```
GET /productos?busqueda=node
```

3. Ordenamiento

Ordenar los resultados según un campo, de forma ascendente o descendente.

Ejemplo:

```
GET /productos?orden=precio_asc  
GET /productos?orden=fecha_desc
```

4. Paginación

Controlar cuántos resultados se devuelven por página y desde qué punto.

Ejemplo:

```
GET /productos?pagina=2&limite=10
```

Devuelve los productos del 11 al 20.

5. Combinación de filtros

Podés combinar varios parámetros:

```
GET /productos?categoria=libros&orden=precio_asc&limite=5
```

Esto permite búsquedas específicas sin necesidad de crear rutas nuevas para cada combinación.

Ejemplo básico de Express que use `req.query`.

```
app.get('/productos', (req, res) => {
  const { categoria, orden } = req.query;
  // Aquí podrías filtrar una base de datos o array según esos
  criterios
  res.send(`Filtrando productos por categoría: ${categoria}, orden:
  ${orden}`);
});
```

HATEOAS (Hypermedia As The Engine Of Application State)

HATEOAS es un principio opcional pero muy poderoso de REST. Consiste en incluir **enlaces de navegación** dentro de las respuestas de la API, permitiendo que el cliente explore los recursos disponibles sin necesidad de saber todas las rutas de antemano.

¿Qué aporta HATEOAS?

- Mejora la **autodescripción** de la API.
- Facilita el desarrollo de clientes dinámicos.
- Reduce la necesidad de documentación externa.

Ejemplo práctico

Supongamos que hacemos una petición para obtener un curso:

```
GET /cursos/10
```

Respuesta con HATEOAS:

```
{
  "id": 10,
  "titulo": "Curso de Node.js",
  "descripcion": "Aprende a construir APIs RESTful.",
  "enlaces": [
    {
      "rel": "self",
      "href": "/cursos/10"
    },
    {
      "rel": "autor",
      "href": "/usuarios/5"
    }
  ]
}
```

```

    "rel": "inscripciones",
    "href": "/cursos/10/inscripciones"
  }
]
}

```

Significado de los campos:

- **rel**: describe la relación con el recurso actual.
- **href**: indica la URL para acceder al recurso relacionado.

Esto le da al cliente la posibilidad de **descubrir otras acciones disponibles** sin saberlas de antemano, como si siguiera “hipervínculos” en una web.

Ejemplo de cómo construir esa respuesta desde Node.js.

```

app.get('/cursos/10', (req, res) => {
  const curso = {
    id: 10,
    titulo: 'Curso de Node.js',
    descripcion: 'Aprende a construir APIs RESTful.',
    enlaces: [
      { rel: 'self', href: '/cursos/10' },
      { rel: 'autor', href: '/usuarios/5' },
      { rel: 'inscripciones', href: '/cursos/10/inscripciones' }
    ]
  };
  res.json(curso);
});

```

Cierre

El diseño de APIs RESTful es una habilidad esencial para cualquier desarrollador backend. A través de este manual, abordamos los conceptos fundamentales de REST, sus características arquitectónicas, ventajas y limitaciones, así como las buenas prácticas necesarias para crear APIs claras, mantenibles y escalables.

Aprendiste a identificar los verbos HTTP correctos, nombrar recursos de forma coherente, estructurar respuestas en formato JSON, aplicar paginación y filtrado, y explorar el concepto de HATEOAS para enriquecer la experiencia de los clientes que consumen tu API.

Recordá que una buena API no solo debe funcionar, sino también ser fácil de entender, usar y evolucionar. Seguir estándares como REST permite que tus aplicaciones sean más robustas y colaborativas, además de facilitar la integración con otros sistemas.

Referencias

- Sequelize. (s.f.). Core concepts: Associations. <https://sequelize.org/docs/v6/core-concepts/assocs/>
- Express. (s.f.). Documentación oficial de Express.js. <https://expressjs.com/es/>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Cap. 5). University of California, Irvine. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- REST API Tutorial. (s.f.). RESTful API design – Best practices in a nutshell. <https://restfulapi.net/>
- Mozilla. (s.f.). HTTP status codes. <https://developer.mozilla.org/es/docs/Web/HTTP/Status>
- Postman. (2020, 6 de octubre). API design best practices. Postman Blog. <https://blog.postman.com/>

¡Muchas gracias!

Nos vemos en la próxima lección

